The benefits of agile integration, Part 1: The fate of the ESB

From its origins in the SOA era to the challenges that inspired the search for a better approach

Kim J. Clark

May 01, 2018
(First published February 13, 2018)

While many large enterprises successfully use the enterprise service bus (ESB) pattern, the term is often disparaged in the cloud-native space, and especially in relation to microservices architecture. It is seen as heavyweight and lacking in agility. What has happened to make the ESB pattern appear so outdated, and what should we use in its place? What would lightweight integration look like?

This two-part series explores the approach that modern integration architectures are taking to ensure they can be as agile as the applications that they interconnect. The pattern of the centralized enterprise service bus (ESB) has served its purpose, and still has its place, but many enterprises are exploring a more containerized and decentralized approach to integration architecture.

Part 1 explores the fate of the ESB. We will briefly look at how and why the centralized ESB pattern arose in the era of service-oriented architecture (SOA), but also at the challenges that came as a result of it. We'll also consider where APIs fit into this picture and what relationship there is between all this and microservices architecture. Without a clear understanding of this history, we cannot make confident statements about how to do things better in the future.

> *" SOA has an enterprise scope and looks at how integration occurs between applications. Microservice architecture has an application scope, dealing with how the internals of an application are built. "*

Part 2 will describe a more lightweight and agile integration style, looking at how integration architecture can benefit from the technologies and principles behind microservices in order to ensure that new applications can perform the integration they need at the pace and scale

© Copyright IBM Corporation 2018

of modern innovation. We will also explore how integration could be more fundamentally decentralized in order to enable greater autonomy and productivity to lines of business.

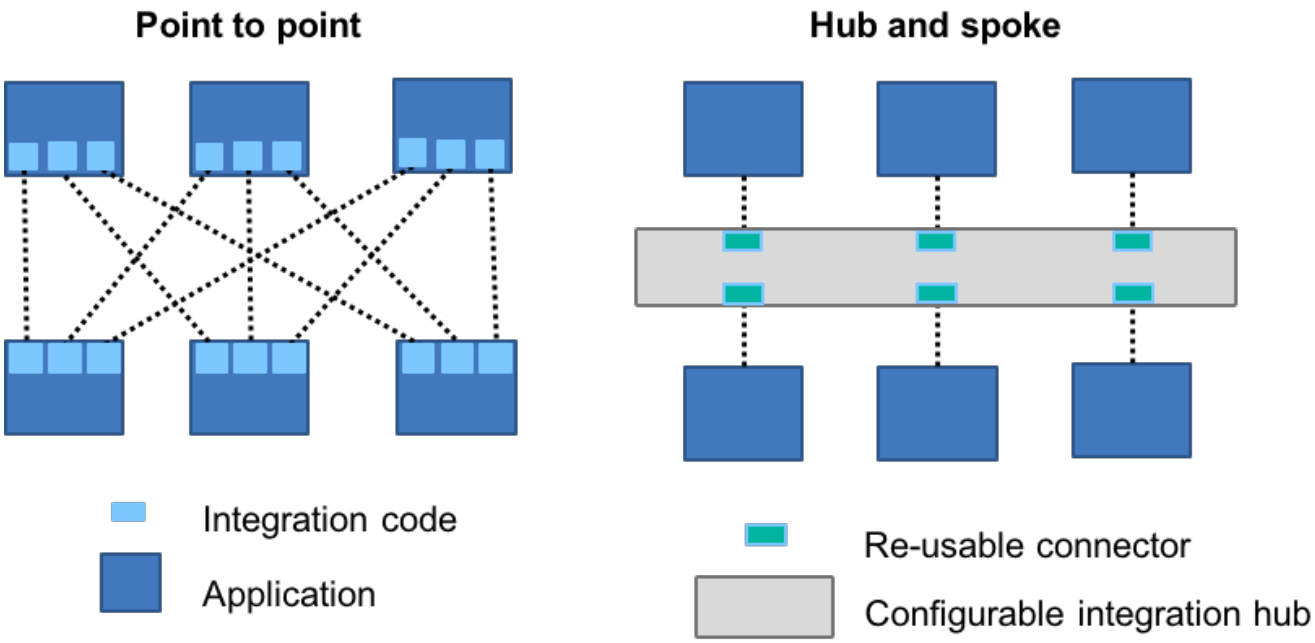
Let's begin by painting a clear picture of what a centralized ESB looks like and where it came from.

Differentiating the ESB pattern from its predecessors

The term ESB is often used very loosely to describe integration runtimes in general, but this is deeply inaccurate, both historically and architecturally.

If you look back to somewhere just before the year 2000, integration was almost exclusively asynchronous, using files and messages to integrate between systems of record. Writing code in or around each system for each and every point-to-point integration was expensive and resulted in a complex web of interactions.

Figure 1. Point-to-point vs. hub-and-spoke patterns



The inevitable result was to introduce an integration hub that sat between all systems, and provided tools that made connectivity much simpler, enabling some amount of re-use of the integration work performed.

It is critically important to note that this primarily asynchronous hub-and-spoke architecture significantly predates the ESB pattern. There are no services being exposed yet. The event-based interaction pattern of hub and spoke is still very common today and, as you'll see later, is perhaps gaining a resurgence due to the increased preference for modern applications to receive data via events. So, to be clear: The hub-and-spoke pattern is not the same as the ESB pattern, even though it may well be implemented using the same integration runtime.

Try IBM Cloud for free

Build your next app quickly and easily with IBM Cloud Lite. Your free account never expires, and you get 256 MB of Cloud Foundry runtime memory, plus 2 GB with Kubernetes Clusters. Get all the details and find out how to get started. And if you're new to IBM Cloud, check out the IBM Cloud Essentials course on developerWorks.

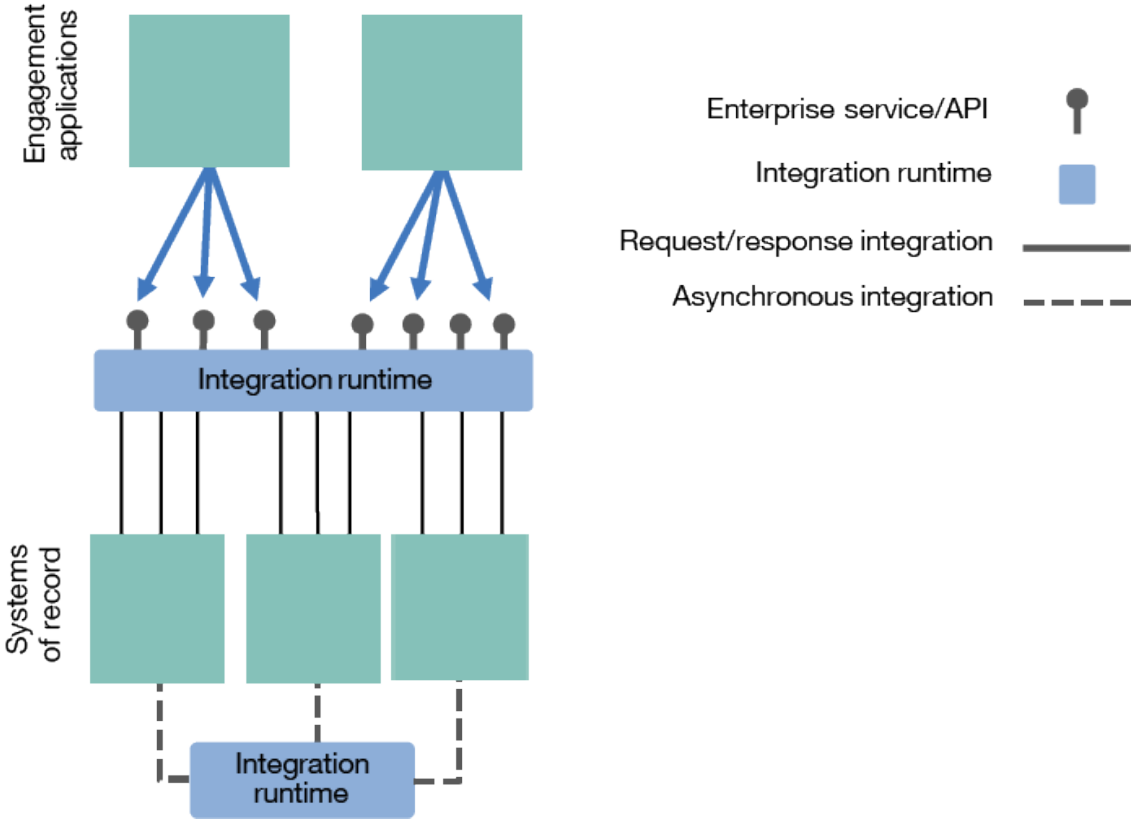
The forming of the ESB pattern

As we started the millennium, we saw the beginnings of the first truly cross-platform protocol for interfaces. The internet, and with it HTTP, had become ubiquitous, XML was limping its way into existence off the back of HTML, and the SOAP protocols for exposing synchronous web service interfaces were just taking shape. Relatively wide acceptance of these standards hinted at a brighter future where any system could discover and talk to any other system via a real-time synchronous remote procedure call, without reams of integration code as had been required in the past.

From this series of events, service-oriented architecture (SOA) was born. The core purpose of SOA was to expose data and functions buried in systems of record over well-formed, simple-to-use, synchronous interfaces, such as web services. Clearly, SOA was about more than just exposing those services, and often involved some significant re-engineering to align the back-end systems with the business needs, but the end goal was a suite of re-usable services. This would enable new applications to be implemented without the burden of deep integration every time, as once the integration was done for the first time and exposed as a service, it could be re-used by the next application.

However, this simple integration was a one-sided equation. We might have been able to standardize the exposure protocols and data formats, but the back-end systems of record were typically old and had antiquated protocols and data formats for their current interfaces. Something needed to mediate between the old system and the new cross-platform protocols.

Figure 2. Synchronous exposure pattern



This *synchronous* exposure pattern via web services was what the enterprise services bus (ESB) term was introduced for. It's all in the name—a centralized "bus" that could expose web "services" across the "enterprise." We already had the technology (the integration runtime) to connect to the back-end systems, coming from the hub-and-spoke pattern. These integration runtimes could simply be taught to expose integrations synchronously via SOAP/HTTP, and we'd have our ESB.

A common source of confusion around the term ESB comes from the fact that at this stage, there was only one component implementing the pattern—the integration runtime. As a result, that integration runtime was often simply referred to as an ESB. Although the integration runtime was in fact performing two separate patterns (hub and spoke, and service exposure), these looked similar enough on an architectural diagram to be merged into a single thing. From that point on, the term ESB was used indiscriminately to refer to the integration runtime itself, regardless of what pattern it was performing.

What went wrong for the centralized ESB pattern?

SOA turned out to be a little more complex than just the implementation of an ESB, for a host of reasons—not the least of which was the question of who would fund such an enterprise-wide program. Implementing the ESB pattern itself also turned out to be no small task.

The ESB pattern often took the "E" in ESB very literally, and implemented a single ESB infrastructure for the whole enterprise, or at least one for each significant part of the enterprise. Tens or even hundreds of integrations might have been installed on a production server cluster,

and if that was scaled up, they would be present on every clone within that cluster. Although this heavy centralization isn't required by the ESB pattern itself, it was almost always present in the resultant topology. There were good reasons for this, at least initially: Hardware and software costs were shared, provisioning of the servers only had to be performed once, and due to the relative complexity of the software, only one dedicated team of integration specialists needed to be skilled up to perform the development work.

The centralized ESB pattern had the potential to deliver significant savings in integration costs, if interfaces could be re-used from one project to the next (the core benefit proposition of SOA). However, coordinating such a cross-enterprise initiative and ensuring that it would get continued funding—and that that funding only applied to services that would be sufficiently re-usable to cover their creation costs—proved to be very difficult indeed. Standards and tooling were maturing at the same time as the ESB patterns were being implemented, so the implementation cost and time for exposing a single service were unrealistically high.

Often, line-of-business teams that were expecting a greater pace of innovation in their new applications became increasingly frustrated with SOA, and by extension the ESB pattern.

Some of the challenges of a centralized ESB pattern were:

- Deploying changes to the interfaces could potentially destabilize other unrelated interfaces, so complex regression testing across a wide range of interfaces was often needed.
- Runtimes were substantial in size due to the number of integrations they contained, so starting and stopping them was highly undesirable. They had to be kept running and patched live wherever possible. This made it hard to track server configurations, and therefore hard to replicate environments for testing and diagnosis, and caused great resistance to adding changes.
- Creating topologies with high availability and disaster recovery for these large servers was costly. Scaling up required up-front planning, and additional servers were expensive.
- Applying the latest middleware fixes and features introduced in new versions was risky, as it could affect existing integrations, so servers typically ran many versions back. This required integration developers to create work-arounds for features that were otherwise available in newer versions.
- The integration specialist teams knew the integration tooling, but often didn't understand the applications they were trying to expose, which added further lead time to the implementation cycle.
- Integration was still a complex field. Few systems exposed good interfaces, so deep skills were required. Only a small handful of integration specialists could be trusted to create, maintain, and administer integrations. In order to pool these specialist resources, they often formed separate "SOA" teams, with strict procedures for receiving work in a waterfall style, introducing a separate dependency to any application development project.
- Service discovery was immature at that time. There were very few options that integrated a service registry with the runtime component. This led to one of two outcomes: Either documentation was stored separately and became quickly outdated, or documentation wasn't

stored at all and each re-use required human-to-human interaction, which eroded the time-to-market promise of re-use.

The result was that creation of services by this specialist SOA team became a bottleneck for projects rather than the enabler that it was intended to be. This typically gave by association the centralized ESB pattern a bad name.

Formally, as we've described, ESB is an architectural pattern that refers to the exposure of services. However, as mentioned above, the term is often over-simplified and applied to the integration engine that's used to implement the pattern. This erroneously ties the static and aging centralized ESB pattern with integration engines that have changed radically over the intervening time.

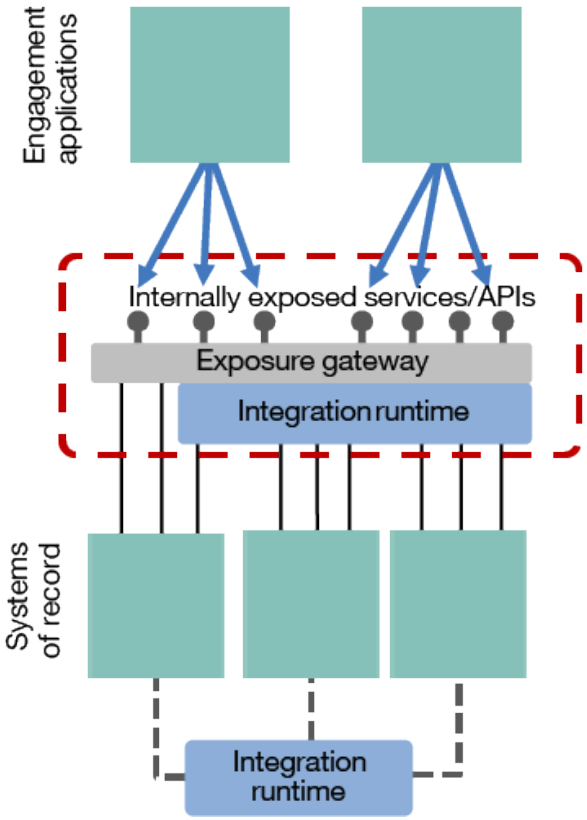
Integration engines of today are significantly more lightweight, easier to install and use, and can be deployed in ways unimaginable at the time the ESB pattern was born. Let's take a look at how these modern runtimes enable completely different architectural patterns that are more lightweight and decentralized.

Supplementing the ESB pattern with a formal exposure gateway

Exposure of request/response-based services is the key differentiator between the ESB pattern and the more event-driven hub-and-spoke pattern that preceded it. SOAP-style RPC interfaces proved complex to understand and use, and simpler and more consistent RESTful services exposed using JSON/HTTP became a popular mechanism for exposure. But the end goal was the same: to make functions and data available via standardized interfaces so that new applications could be built on top of them more quickly.

With the broadening usage of these service interfaces, both within and beyond the enterprise, more formal mechanisms for exposure were required. It quickly became clear that simply making something available over a web service interface, or latterly as a RESTful JSON/HTTP API, was only part of the story. That service needed to be easily discovered by potential consumers, who needed a path of least resistance for gaining access to it and learning how to use it. Additionally, the providers of the service or API needed to be able to place controls on its usage, such as traffic control and an appropriate security model.

Figure 3. Enhancing the ESB pattern with a separate service exposure gateway



Some of these capabilities could be introduced into the integration runtime, but due to the heavyweight and complex nature of the centralized ESB pattern, this meant adding even more responsibilities to the already overburdened ESB team. A common alternative approach was to separate the role of service/API exposure out into a separate gateway.

These exposure capabilities evolved into what is now known as API management, and enabled simple administration of service/API exposure. The gateways could also be specialized to focus on API management-specific capabilities, such as traffic management (rate/throughput limiting), encryption/decryption, redaction, and security patterns. The gateways could also be supplemented with portals that describe the available APIs and enabling self-subscription in order to use the APIs, and provisioning analytics for both users and providers of the APIs.

Increasingly, more modern systems of record already provided an HTTP-based interface that only needed controlled exposure using the exposure gateway. The integration runtime was only required when more complex integration took place, such as more unusual protocols, data formats, compositions of multiple requests—or perhaps in cases where transactionality was needed.

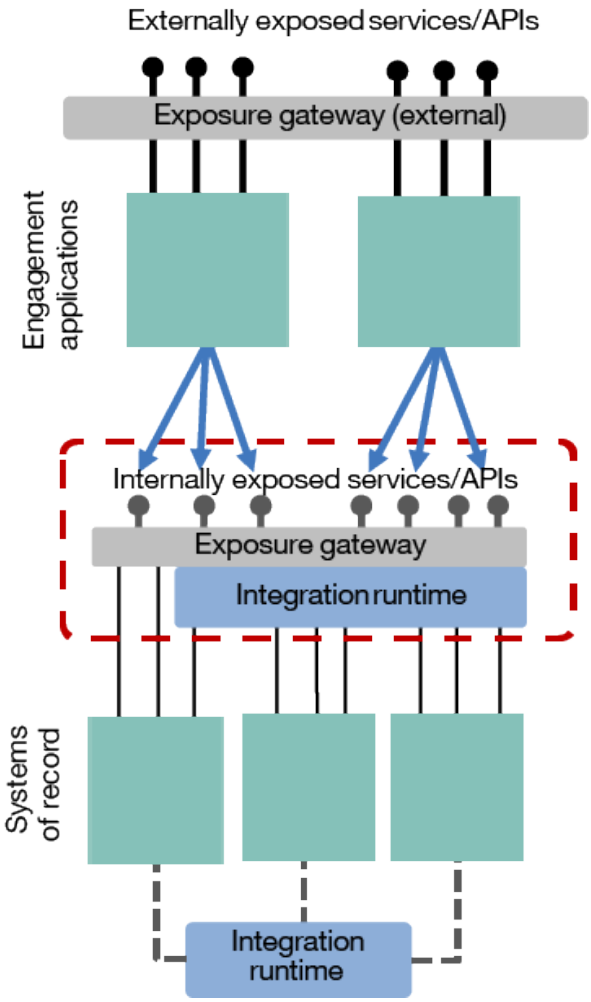
The introduction of an API management layer led to the obvious question: What now is the ESB? Many had come to see the integration runtime and the ESB pattern as one and the same. But in fact, if the ESB pattern is all about exposing services and APIs, then the boundaries of the pattern

really include both the integration runtime and the exposure gateway, and in some cases just the gateway. However, due to the ESB's incorrect association with the integration runtime, we have to accept that this is not how the ESB term is typically used.

APIs spread outside the enterprise boundary

Once the mechanisms for effectively exposing APIs had matured, it became clear that they could also be exposed outside the enterprise. Initially, this was done to create the *back-end-for-front-end (BFF)* pattern that's still prevalent for mobile applications and single-page web applications. In this pattern, APIs were created specifically for the front-end application and perfectly suited to its needs with rationalized data models, ideal granularity of operations, specialized security models, and more. Soon it became clear that APIs could be exposed more broadly to enable any partner to write applications, opening up a whole new set of opportunities for collaboration.

Figure 4. The external service/API gateway and the beginnings of the API Economy



It is hard to pin down the exact historical order here, as it varies by enterprise. The concepts of API management for some enterprises *began* with external exposure, and were only brought

inside to supplement the ESB pattern later. Whichever the sequence, external APIs have become an essential part of the online persona of many companies, and are at least as important as its websites and mobile applications.

Logically, the exposure of APIs outside the enterprise is just an extension of the ESB pattern, with more focus on the gateway and aspects such as security, discovery, and self-administration. I've covered the technical differences between APIs and SOA services in-depth in a previous article. For example, it is immediately obvious that the APIs are being used by consumers and devices that may exist anywhere from a geographical and network point of view. As a result, it is necessary to design the APIs differently to take into account the bandwidth available and the capabilities of the devices used as consumers. However, there are non-technical aspects to the differences, too. You should not underestimate the difference in the business objectives of the exposed APIs. External API exposure is much less focused on re-use, in the same way that internal services were in SOA, and more focused on creating services targeting specific niches of potential for new business. APIs provide an enterprise with the opportunity to radically broaden the number of innovation partners that it can work with (enabling crowd sourcing of new ideas), and they play a significant role in the disruption of industries that is so common today. This realization caused the birth of what we now call the API Economy, and it is a well-covered topic in its own right.

The main takeaway here is that this progression exacerbated an already growing divide between the older traditional *systems of record* that still perform all the most critical transactions fundamental to the business, and what became known as the *systems of engagement*, where innovation occurred at a rapid pace, exploring new ways of interacting with external consumers. This resulted in **bi-modal IT**, where new decentralized, fast-moving areas of IT needed much greater agility in their development, and led to the invention of new ways of building applications using, for example, microservices architecture.

The rise of microservices

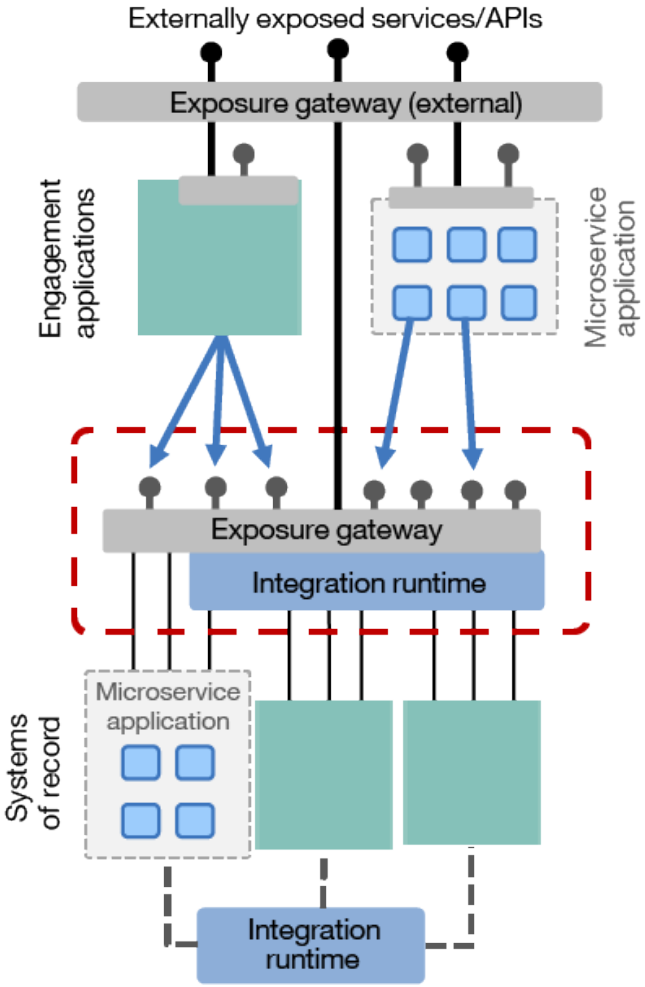
Earlier, I covered the challenges of the heavily centralized integration runtime—hard to safely and quickly make changes without affecting other integrations, expensive and complex to scale, etc. Sound familiar? It should. These were exactly the same challenges that application development teams were facing at the same time: bloated, complex application servers that contained too much interconnected and cross-dependent code, on a fragile cumbersome topology that was hard to replicate or scale. Ultimately, it was this common paradigm that led to the emergence of the principles of microservices architecture. As lightweight application servers such as IBM WAS Liberty were introduced—servers that started in seconds and had tiny footprints—it became easier to run them on smaller virtual machines, and then eventually within container technologies such as Docker.

In order to meet the constant need for IT to improve agility and scalability, a next logical step in application development was to break up applications into smaller pieces and run them completely independently of one another. Eventually, these pieces became small enough that they deserved a name, and they were termed **microservices**. Perhaps a better name would be **microservice components** as the term often causes confusion (especially in integration circles, as explained in previous articles and videos), but "microservices" has now become pervasive.

If you take a closer look at microservices concepts, you will see that it has a much broader intent than simply breaking things up into smaller pieces. There are implications for architecture, process, organization, and more—all focused on enabling organizations to better use cloud-native technology advances to increase their pace of innovation.

However, focusing back on the core technological difference, these small independent microservice components can be changed independently to create greater agility, scaled independently to make better use of cloud-native infrastructure, and managed more ruthlessly (see my article "Cattle not pets") to provide the resilience required by 24/7 online applications.

Figure 5. Microservices architecture: A new way to build applications



In theory, these principles could be used anywhere. Where we see them most commonly is in the systems of engagement layer, where greater agility is essential. However, they could also be used to improve the agility, scalability, and resilience of a system of record—or indeed anywhere else in the architecture, as you will see shortly.

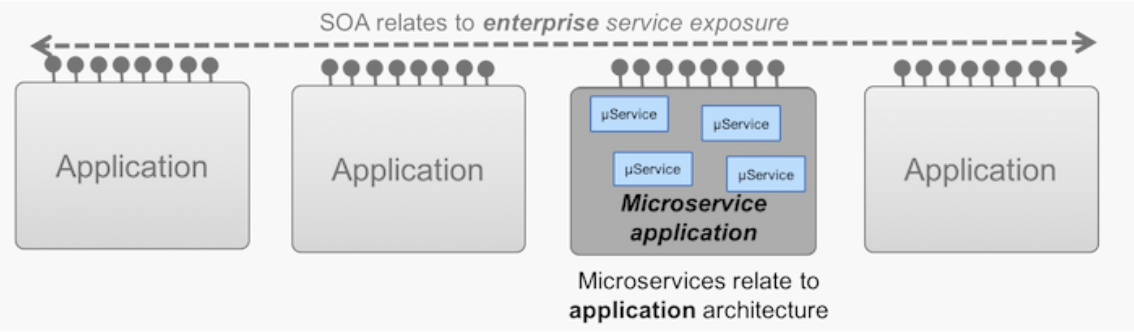
Without question, microservices principles can offer significant benefits under the right circumstances. However, choosing the right time to use these techniques is critical, and getting the design of highly distributed components correct is non-trivial. You need only read Martin

Fowler's opinions on this to see the dilemma unfold. At the end of the day, deciding the shape and size of your microservice components is only part of the story; there is an equally critical set of design choices around the extent to which you decouple them, and you need to constantly balance practical reality with aspirations for microservices-related benefits. Although decoupling is fundamental to microservices, that doesn't mean full decoupling is always good microservice design. Good design is always a compromise. In short, *your microservice-based application is only as agile and scalable as your design is good, and your methodology is mature.*

A comparison of SOA and microservice architecture

It is tempting to compare microservices architecture with SOA, not least because they share many words in common. However, as you will see, this comparison is misleading at best, since the terms apply to two very different scopes.

Figure 6. SOA is enterprise scoped, microservices architecture is application scoped



Service-oriented architecture is an *enterprise-wide* initiative to create re-usable, synchronously available services and APIs, such that new applications can be created more quickly incorporating data from other systems.

Microservices architecture, on the other hand, is an option for how you might choose to write an *individual application* in a way that makes that application more agile, scalable, and resilient.

It's critical to recognize this difference in scope, as some of the core principles of each approach could be completely incompatible if applied at the same scope. For example:

- **Re-use:** In SOA, re-use of integrations is the primary goal, and at an enterprise level, striving for some level of re-use is essential. In microservices architecture, creating a microservice component that is re-used at runtime throughout an application results in dependencies that reduce agility and resilience. Microservice components generally prefer to re-use code by copy and, as you'll see shortly, accept data duplication in order to improve decoupling between the components.
- **Synchronous calls:** The re-usable services in SOA are exposed across the enterprise using predominantly synchronous protocols such as RESTful APIs. However, *within* a microservice application, synchronous calls introduce real-time dependencies, resulting in a loss of resilience, and also latency, which impacts performance. *Within* a microservice application,

interaction patterns based on asynchronous communication are preferred, such as event sourcing where a publish subscribe model is used to enable a microservice component to remain up to date on changes happening to the data in another component.
- **Data duplication:** A clear aim of exposing services in an SOA is for all applications to get hold of, and make changes to, data directly at its primary source, which reduces the need to maintain complex data synchronization patterns. In microservice applications, each microservice ideally has local access to all the data it needs to ensure its independence from other microservices, and indeed from other applications—even if this means some duplication of data in other systems. Of course, this duplication adds complexity, so it needs to be balanced against the gains in agility and performance, but this is accepted as a reality of microservice design. With data duplication so prevalent in microservices, there is still the need to have an agreed-upon source of truth for each type of data.

So, in summary, SOA has an *enterprise* scope and looks at how integration occurs between applications. Microservice architecture has an *application* scope, dealing with how the internals of an application are built. This is a relatively swift explanation of a much more complex debate, which is thoroughly explored in a separate article, but it provides the key concepts we need here.

Making use of microservice principles in integration

So if it makes sense to build applications in a more granular fashion, why couldn't you apply this idea to integration, too? You could break up the enterprise-wide centralized ESB component into smaller, more manageable, and dedicated components. Perhaps you could go so far as to have one integration runtime for each interface you expose.

This pattern is referred to as **agile integration architecture (AIA)**, although note that it was briefly also known as *lightweight integration*. These terms differentiate it from full microservices application architecture, and also to distinguish it from the ESB term, which is strongly associated with the more cumbersome centralized integration architecture.

In Part 2, I'll describe agile integration architecture in more detail, looking at how integration architecture can benefit from the technologies and principles behind microservices. This ensures that new applications can perform the integration they need at the pace and scale of modern innovation.

Acknowledgements

Thank you to the following people for their input and review of the material in this article: Andy Garratt, Nick Glowacki, Rob Nicholson, Brian Petrini, and Claudio Tagliabue.

Related topics

- Integration architecture: Comparing web APIs with service-oriented architecture and enterprise application integration
- Microservices, SOA, and APIs: Friends or enemies?
- Cattle not pets: Achieving lightweight integration with IBM Integration Bus
- Martin Fowler: Microservices and the First Law of Distributed Objects
- Try IBM Cloud for free

© Copyright IBM Corporation 2018
(www.ibm.com/legal/copytrade.shtml)
Trademarks
(www.ibm.com/developerworks/ibm/trademarks/)